

DSE-4
PROJECT WORK

Project Name: Hotel Administrative System

Submitted By:

Avijit Bera

Soumen Pradhan

Suman Bar

Amit Ranjan Paria

Submitted To:

Department of Computer Science & Application

Prabhat Kumar College

Contai Purba Medinipur

Table Of Contents

Table Of Contents	1
1) Project Proposal	2
1) Introduction:	3
2) Goal of the project:	3
3) Issues/Limitation:	4
4) Objectives:	4-5
5) Feasibility Analysis:	5-6
6) Cost-Benefit Analysis:	6-7
7) Types of stakeholders(users) of the proposed system:	7
8) Functionality grouping according to the types of users:	7-8
9) Non-Functional Requirements (NFRs):	8-10
10) Overall Constraints:	10
2) Hotel Administrative System Entity Relationship Diagram	11
3) Hotel Administrative System GANTT Chart	12-15
4) Use Case Diagram/Model	16-19
5) Module	20-31

1) Project Proposal

1) Introduction:

Hotel Administrative System is a desktop application where Employee can book a hotel room. Managers can manage rooms, book rooms for new customers, and also solve other issue. And the admin of this application can maintenance for their application and can also see database of the hotel.

In this way, the hotel Administrative will be easier to handle with this application than they used to keep track of all details manually in their ledger. The customers, managers, and Employee can be happier with this new system.

2) Goal of the project:

We plan to create an application that can manage all the necessary information and work for the hotel. So that the suffering of each Employee, manager, and admin would decrease significantly.

- ★ In this project, the customers would be provide their personal details and provide their requirement of room and meal.
- ★ The managers would be able to manage their hotel room details.
- ★ The managers would also be able to book any customers' hotel rooms.

- ★ The managers would be able to check out and give bill memos to their customers.
- ★ The Employee would be able to create, update and delete customer details.
- ★ The Employee would also be able to book any customers' hotel rooms.
- ★ The admin would also be able to maintenance of application and maintain database.

3) Issues/Limitation:

- Saving information in a ledger makes things complex to handle.
- Customers need to come in to book hotel rooms.
- It works on windows operating system.
- No way to pay online bill on this application.
- No receipt print option.

4)Objectives:

- Using a MySQL database can save information in an organized way and we can fetch data into our application by users' needs.
- Add a feature so that the Employee and manager can view empty rooms from his application.

- The application should have a way to calculate the total bills of a customer till check out and generate a bill based on their room type, meal type and total no of stay day.
- There should be a system of storing employee information in a way so that we can update, create and delete the information of the employees.
- Customer details searching system based on customer Mobile number and Ref id.
- Room book searching system based on customer mobile number and room no.

4) Feasibility Analysis:

Technical Feasibility:

- Our application can be easily run from any computer.
- Our application will run and work efficiently on any hotel computer.
- If the application needs to be updated in the future it can be easily updated.
- This application has long-term support availability.

Economic Feasibility:

As our application is made for running on windows computers it can be runnable on any computer. And its economic cost would be higher at the software development stage. As the hotel had a budget to build this application, it would be made within this budget.

Concerning the economic budget of the hotel, we are making this project budget-friendly and it will not increase until there are more software changes in the future.

Operational Feasibility:

This application will change the future of users and the hotel Administrative system. Because it's totally made in a way that software can be run easily on a PC. Besides, it has a user-friendly User Interface so every user will have no issue running this application.

We also offer our future service to bring more updates to this application. Which will ensure this software's long-term stability. However, the way the software is right now users will have a smooth software experience.

8) **Cost-Benefit Analysis:**

Tangible cost: Cost of system analysts' time, and software development time are the tangible costs here.

Intangible cost: Costs of internet, computers and electricity are the Intangible costs. Where the hotel has to pay for this as long as they want to use their software.

Tangible benefit: If the project is finished within the time then it's software development cost would be minimized and it would be launched before the deadline of making this software. This will reduce the hotel customers, managers and employees suffer more early and hotel owners will be also happy to get their software within time.

Intangible benefit: Though the software has an Intangible cost which is mandatory to run this software it will make a positive impact on the hotel. The customers and hotel managers both will be happy and they may get positive feedback from their Employee by using this software. Besides, hotel customers may increase after they start using this software.

9) Types of stakeholders(users) of the proposed system:

- 1) Employee
- 2) Manager
- 3) Admin

10) Functionality grouping according to the types of users:

The new system will perform many kinds of operations to make customers, managers, and employees' lives easier. With this,

- **Employee:** The Employee will be able to book Customer hotel rooms, they would be able to Modify the Customer details and Room book details of this hotel service. The Employee will generate bill for customer requirement.

- **Manager:** The managers will be able to see the room details if there are any empty rooms or not with this software. They would also be able to check in their customers to any hotel rooms and check out from any hotel rooms. This application will generate check-out bills for customers, which will reduce the bill calculating pain for managers. Finally, there will be a section for unsolved problems which they can solve in the future to make their hotel service better.
- **Admin:** The admin will be able to handle all database of this hotel service. The admin will also be able to solve any problem of back end work.

Non-Functional Requirements (NFRs):

Efficiency requirement:

- This application will be programmed in a way that users will have a smooth experience.
- This application will load the first page within 1 second.
- The average load time of every page in this application is under 1 second which is pretty efficient.

- This application will connect and fetch data from the database pretty quickly.

Portability requirements:

- This application will run on any version of Windows computer.
- It can be run on any low-configuration computer with the windows operating system.
- Anyone can take this application installer to any other computer through any storage device and install this application.

Usability requirements:

- The user interface in this application is so user-friendly that they can easily understand every option in the application.
- As to make the User Interface FXML is being used which is super lightweight and can load within 100 milliseconds. So users will experience the lightning-fast UI.

Security requirements:

- ➔ As it is a desktop application and Python is being used so the application will be made with proper use of Python security. Because Python is the most secure language.
- ➔ The application will be used in such a way that every Employee, manager, and admin will have a password to use this application. So there is no way that any anonymous user can use this application.
- ➔ Also, the application has used the database MySQL server in a secure way that no one except the application developer will be able to know where the server is. So all the users will feel secure using this application.

Availability requirements:

- This application will run 24 hour a week and the server will serve this application without any delay as the server has backup power to continue through electricity is gone.

Scalability requirements:

- ➔ This application can store up to 500 MB of data in the database of the users at the same time.
- ➔ This application can handle any kind of exception that may occur from users.

Budget requirements:

- The initial budget for this application is 5 lakh Rupee. As the system analysis team analyzed the hotel business requirements and their demand. We found that it is possible to make this application within this budget and make this project successful.

11) **Overall Constraints:**

- No way to pay online
- No way to order food
- No android, IOS, or web application
- Computer operating system should be windows

2) Hotel Administrative System Entity Relationship Diagram

There are 5 entities used to make this ER Diagram.

The entities are

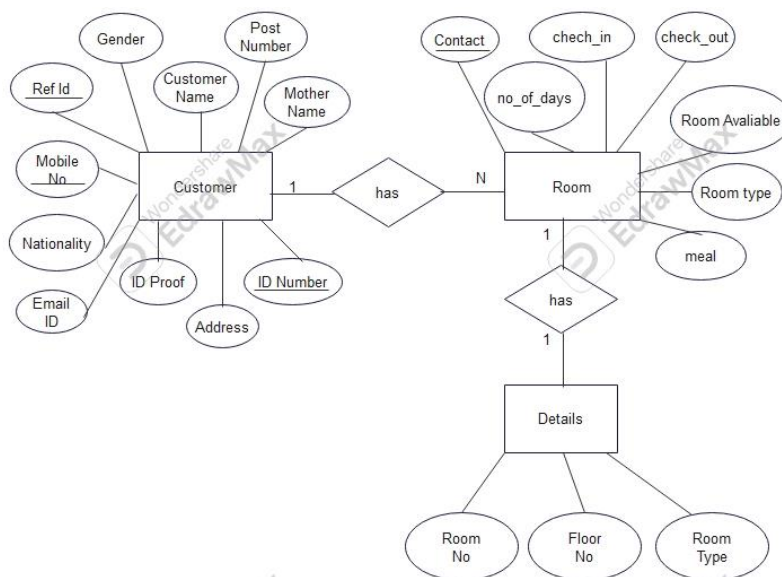
1. Customer
2. Room book
3. Room details

Customer Info: In this entity, there is all information about the customer's name, Ref Id, mother name, gender, pin code, phone number, email id, nationality, Id proof & number, address.

Room Book : In this entity, there is all information about the customer's check-in date, check-out date, meal, stay day, the room no was checked in, and the total amount of money the customer needs to pay after room entry. These kinds of information will be available here.

Room details: In this entity, there is all information about the room no, room type, and room capacity. Also, there is a room status attribute. With which we can see if there are any available rooms or not.

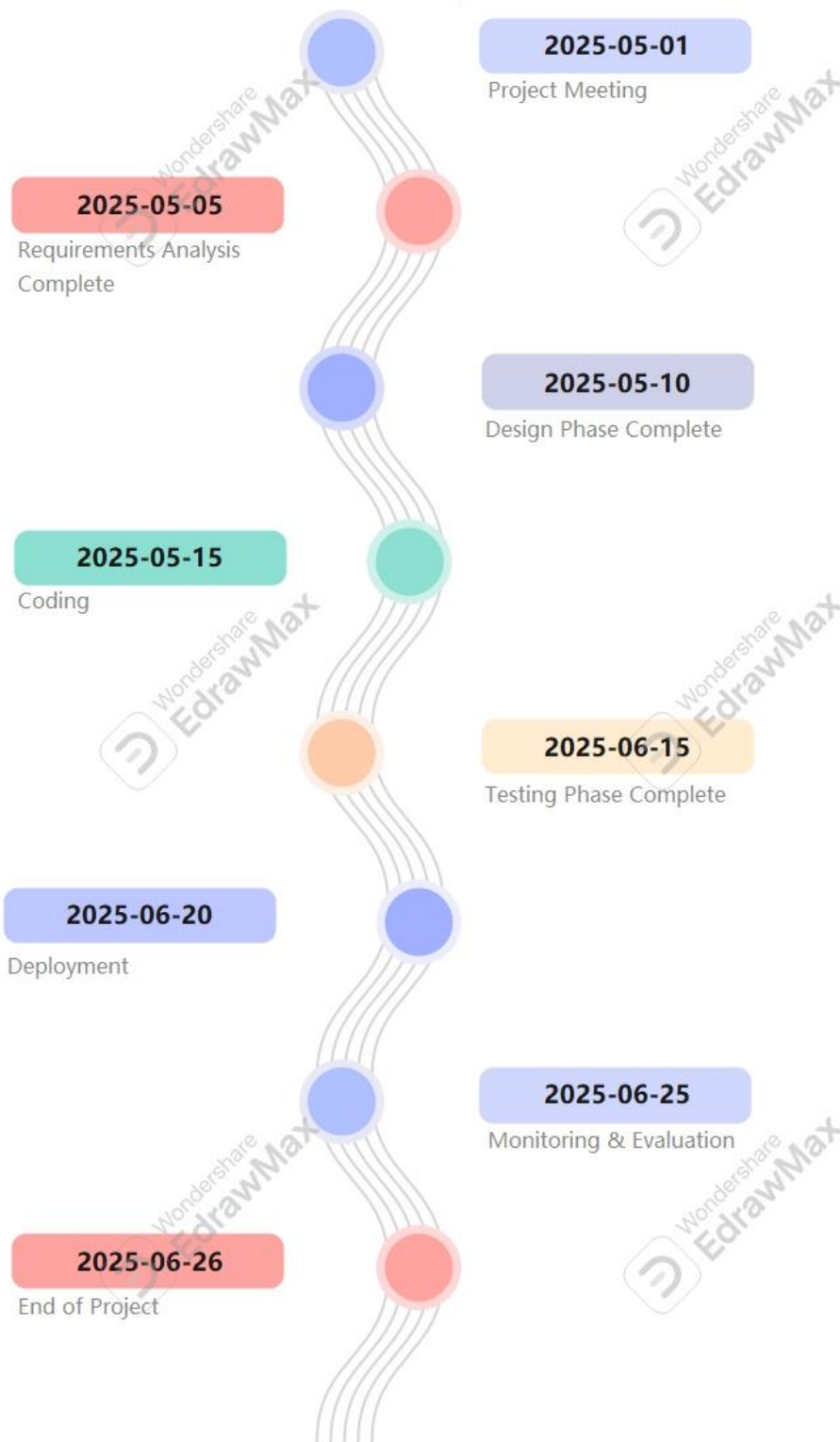
In these entities, there are many attributes including



3) Hotel Administrative System GANTT Chart

Activity	Predecessor	Duration (Days)
Kick-Off Meeting	None	4
Analysis	Kick-Off Meeting	5
Design	Analysis	5
Coding	Design	30
Testing	Coding	5
Deployment	Testing	5
Monitoring & Evaluation	Deployment	1

Timeline of Key Events



So, in this GANTT chart there are 7 types of activities. These are kick-off meetings, analysis, design, coding, testing, release, deployment, monitoring & evaluation of the software.

- 1) Meeting:** First of all, we started our project with our "SUNWAVE" group members through a meeting. There we discussed how we should start the project, how the activity duration should be, how we will do the feasibility study, who will do the design, coding, and testing, etc. That meeting took about 4 days, which started from May 1 to May 5.
- 2) Analysis:** After finishing the meeting we started to do different types of analyses of the project. Where we used to do case studies, finding issues, problems, project goals, feasibility analyses, and cost-benefit analyses. We also used to find out how many types of users will be in our project and what will be their functionality. What will be the non-functional requirements to maintain our software. So our analysis lasted about 5 days. Which started on May 5th and ended on May 10th.
- 3) Design:** So in the design part first of all we made a prototype design of our software so that we have the whole software mind map. Then by following the prototype we tried to build the whole software user interface which is user-friendly. In this design process, we took about 5 days to finish. Which started from May 10, 2025 to May 15, 2025.
- 4) Coding:** In the coding part, we started to do this from May 15, 2025, to June 15, 2025. Which lasted about 30 days. These days, we used to make all functionality work. Like, we made our user interface before and our task was to make those user interfaces

work. So we needed to implement these interface controller parts which are totally related to programming with Python. So by the hard work of our team, we finally finished this part after 30 days.

- 5) Testing:** After finishing the coding part. We needed to make sure if all functionality of the software works or not, if the codes are clean and secured or not. So to make sure of these things, we tested our software as well as the coding part again and again. Then if we find any errors those were noted so that would be fixed later. In this testing part, it took almost 5 days to finish. Which started on June 15, 2025 and lasted till June 20, 2025.
- 6) Deployment:** After ensuring that the software works totally fine and it fills all the requirements of the client. We started writing all the manuals and documentation about the software which would be used to run this software so that everyone can have basic knowledge about all of the functionalities. We also packed the software in such a way so that it would be easily installed on any windows computer.
- 7) Monitoring & Evaluation:** After deployment we monitoring this application then we modify some functionality and evaluation some feature. After this we re deployment this application.

4) Use Case Diagram/Model

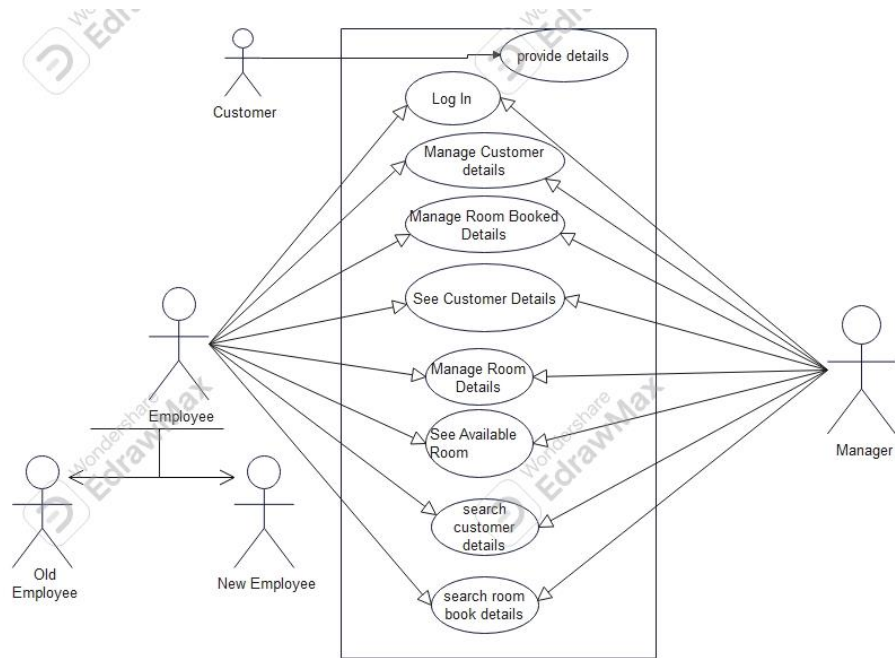
- ❖ contains the following symbols:
 - ✓ Actor symbols
 - ✓ Use Case symbols
 - ✓ System Boundary and
 - ✓ Connecting lines
- ❖ cases are now being used regardless of the approach to systems development.
- ❖ helps us to
 - ✓ specify the system context and design it from the end user's perspective.
 - ✓ gather/capture the requirements of the system.
 - ✓ depict the interaction between the end-user and the system.

Components of Use Case Diagram (UCD) are as follows:

- ❖ Actors
 - ✓ The users of the system
- Use Cases
 - ✓ Functionalities of the system
- ❖ System Boundary
 - ✓ Overall Scope of the system

❖ Communication Lines

✓ Relationships between various other components.



Here in this Use Case Diagram as this is a Hotel Administrative System there we used 3 kinds of Actors. They are Customer, Employee, and Admin .

Actor/user : the users of the system.

Customer, Employee and Admin .

❖ For customer's perspective :

Customer's have some access for this diagram.

sign up.

Provide personal information

Provide requirement of room and meal

❖ For employee's perspective :

Check in available room

Logs in

See customer information

Manage customer details

Manage room book details

❖ For admin's perspective :

Logs in

Adds new Room details

See customer information

❖ Use cases:

Functionality of the system.

In this Hotel Administrative System's UCD, there are so many use cases.

They are ,

Check in available room

Logs in

See profile information

Adds new room

Check out

Adds new employee

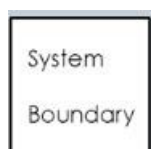
Sees room booked information

Sees customer's information

System boundary: overall scope of the system

- ❖ is potentially the entire system as defined in the requirements document.
- ❖ maybe indicated by each module for large and complex systems.

For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc. can form a system boundary for use cases specific to each of these business functions. The entire system can span all of these modules depicting the overall system boundary. Symbol is ,



- ❖ Communication lines: relationships between various other components.

5) Module

- Login
- Hotel
- Customer Details
- Room Book Details
- Room Details
- About Us

1.login:

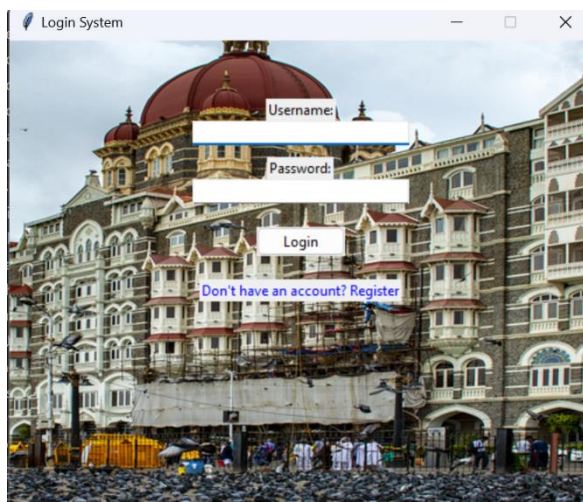
The file implements a simple login and registration system using Python's Tkinter GUI library. It provides a graphical interface for users to log in or register for an account. The main features include:

- **Login Window:**

Users can enter their username and password to log in. Credentials are checked against an in-memory dictionary ([self.users](#)).

- **Registration Window:**

Users can create a new account by providing a username, password, and confirming the password. The system checks for empty fields, password match, and duplicate usernames.



- **Dashboard Launch:**

On successful login, the system attempts to open the hotel Administrative dashboard by importing and launching the [HotelAdministrativeSystem](#) class from the [hotel](#) module.

- **GUI Elements:**

The interface uses Tkinter widgets such as [Label](#), [Entry](#), [Button](#), and [Toplevel](#) for windows and forms. The background image is loaded using PIL.

- **Message Box**

- 1.Input Validation:**

- [messagebox.showwarning\("Input Error", "Please enter both username and password"\)](#)

- Shown when the user tries to log in without entering both fields.

- [messagebox.showwarning\("Input Error", "All fields are required"\)](#)

- Shown during registration if any field is left empty.

- 2.Login Feedback:**

- [messagebox.showinfo\("Login Success", f"Welcome, {username}!"\)](#)

- Shown when the user logs in successfully.

- [messagebox.showerror\("Login Failed", "Invalid username or password"\)](#)

- Shown when the login credentials are incorrect.

- 3.Registration Feedback:**

- [messagebox.showerror\("Error", "Passwords don't match"\)](#)

- Shown if the password and confirm password fields do not match during registration.

- [messagebox.showerror\("Error", "Username already exists"\)](#)

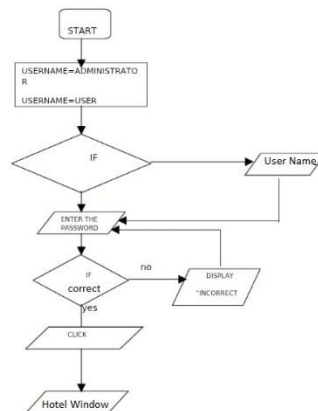
- Shown if the chosen username is already taken.

- [messagebox.showinfo\("Success", "Registration successful!"\)](#)
Shown when a new user is registered successfully.

4.System Errors:

- [messagebox.showerror\("Error", "Hotel system not found."\)](#)
Shown if the hotel dashboard module cannot be loaded after login.
- These message boxes enhance user experience by providing clear, immediate feedback and guidance during authentication and registration processes.

- Flow Chart:



2)HOTEL:

This file implements the main dashboard for a hotel Administrative system using Python's Tkinter GUI library. The dashboard provides access to various hotel Administrative features through a graphical interface. Key features include:

- **Main Window:**
Displays the hotel name, logo, and several images for branding and aesthetics.

- **Menu Panel:**

A sidebar menu with buttons for:

- **Customer:** Opens the customer Administrative window.
- **Room:** Opens the room booking window.
- **Details:** Opens the room details window.
- **About Us:** Opens an about/report window.
- **Logout:** Allows the user to log out of the system.



- **Navigation:**

Each menu button opens a new window for its respective function by creating a new [Toplevel](#) window and initializing the corresponding class

([Cust Win](#), [roombooking](#), [DeatalisRoom](#), [report](#)).

- **Image Handling:**

Uses the PIL library to load and resize images for the interface.

- **Logout Functionality:**

Prompts the user for confirmation before closing the application.

This file serves as the main interface after a successful login, providing access to all core hotel Administrative modules.

3)CUSTOMER:

This file defines the customer Administrative module for the hotel Administrative system using Python's Tkinter GUI library. It allows users to add, view, update, delete, and search customer records. Key features include:

- **Customer Form:**

A form for entering customer details such as reference number, name, mother's name, gender, PIN code, mobile number, email, nationality, ID proof, ID number, and address.

- **Buttons:**

Buttons for Add, Update, Delete, and Reset operations, each performing the respective database action.

The screenshot shows a Tkinter GUI window titled "Customer". The window is divided into two main sections. The left section, titled "CUSTOMER DETAILS", contains a form for adding customer information. The right section, titled "View Details And Search System", contains a search bar and a table of customer records.

CUSTOMER DETAILS Form:

- Customer Ref: 7231
- Customer Name:
- Mother Name:
- Gender: Male
- PIN Code:
- Mobile No:
- Email ID:
- Nationality: Indian
- ID Proof: Adhar Card
- ID Number:
- Address:

Buttons: Add, Update, Delete, Reset

View Details And Search System:

Search By: Mobile Search Show All

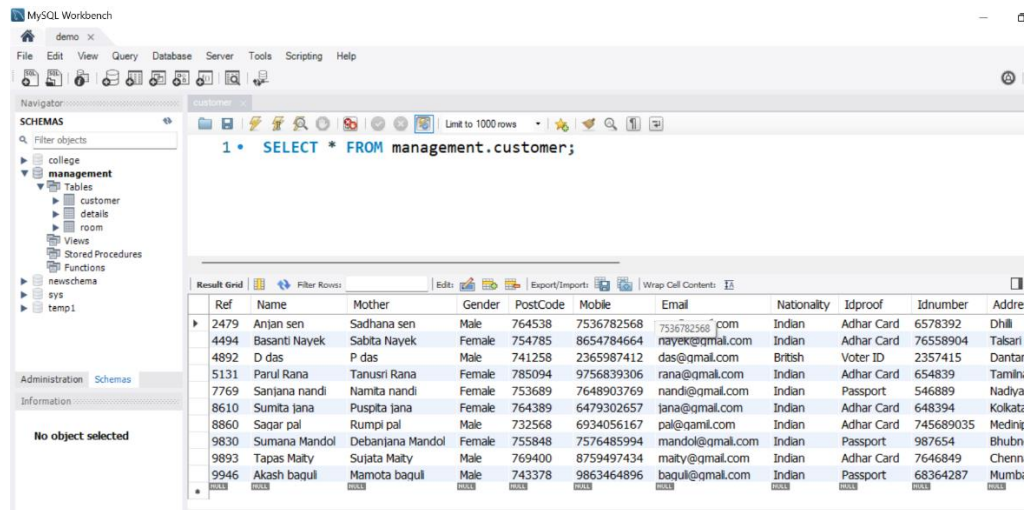
Refer No	Name	Mother Name	Gender	PostCode	Mobile	Email	Nationality	ID
2479	Anjan sen	Sadhana sen	Male	764538	7536782568	sen@gmail.com	Indian	Adhar
4494	Basanti Nayek	Sabita Nayek	Female	754785	8654784664	nayek@gmail.com	Indian	Adhar
4892	D das	P das	Male	741258	2365987412	das@gmail.com	British	Voter I
5131	Parul Rana	Tanusri Rana	Female	785094	9756839306	rana@gmail.com	Indian	Adhar
7769	Sanjana nandi	Namita nandi	Female	753689	7648903769	nandi@gmail.com	Indian	Passpo
8610	Sumita jana	Puspita jana	Female	764389	6479302657	jana@gmail.com	Indian	Adhar
8860	Sagar pal	Rumpi pal	Male	732568	6934056167	pal@gmail.com	Indian	Adhar
9830	Sumana Mandol	Debanjana Mandol	Female	755848	7576485994	mandol@gmail.com	Indian	Passpo
9893	Tapas Maity	Sujata Maity	Male	769400	8759497434	maity@gmail.com	Indian	Adhar
9946	Akash baguli	Mamota baguli	Male	743378	9863464896	baguli@gmail.com	Indian	Passpo

- **Database Integration:**

Connects to a MySQL database (Administrative) to store and manage customer data in the customer table.

- **Data Table:**

Displays all customer records in a table with horizontal and vertical scrollbars. Selecting a row loads the data into the form for editing.



- **Search Functionality:**

Allows searching for customers by mobile number or reference number.

- **Validation and Error Handling:**

Uses message boxes to notify users of errors, successful operations, and confirmation.

- Input Validation and Errors: `messagebox.showerror("Error", "All fields are required", parent=self.root)`. Shown when required fields (like mobile or mother's name) are missing while adding a customer.

- `messagebox.showerror("Error","Please enter mobile number",parent=self.root)`. Shown when trying to update a customer without entering a mobile number.
- Operation Success: `messagebox.showinfo("Success","Customer has been added", parent=self.root)`. Shown when a customer is successfully added to the database.
- `messagebox.showinfo("Update","Customer Details has been updated seccessfully",parent=self.root)`. Shown when customer details are successfully updated.
- Warnings and Exceptions: `messagebox.showwarning("Warning",f"Something went wrong: {str(es)}", parent=self.root)`. Shown if an exception occurs during the add operation, displaying the error message.
- Delete Confirmation: `messagebox.askyesno("Hotel Administrate system","Do you want delete this customer",parent=self.root)`
Asks the user for confirmation before deleting a customer record.

This file provides a complete CRUD (Create, Read, Update, Delete) interface for managing hotel customer information.

4)Room Book:

This file implements the room booking module for the hotel Administrate system using Python's Tkinter GUI library. It allows users to

manage room bookings, calculate costs, and interact with the underlying MySQL database. Key features include:

- **Room Booking Form:**

A form for entering booking details such as customer contact, check-in and check-out dates, room type, available room number, meal plan, number of days, paid tax, subtotal, and total cost.

- **Dynamic Room Selection:**

Room numbers are dynamically updated based on the selected room type and availability.

- **Meal and Room Type Options:**

Users can select from various meal plans and room types, with pricing logic applied accordingly.

- **CRUD Operations:**

Buttons for Add, Update, Delete, and Reset allow users to manage room booking records in the database.

- **Customer Data Fetch:**

Fetches and displays customer details based on the contact number for quick reference.

- **Booking Table:**

Displays all room bookings in a table with search functionality by contact or room number.

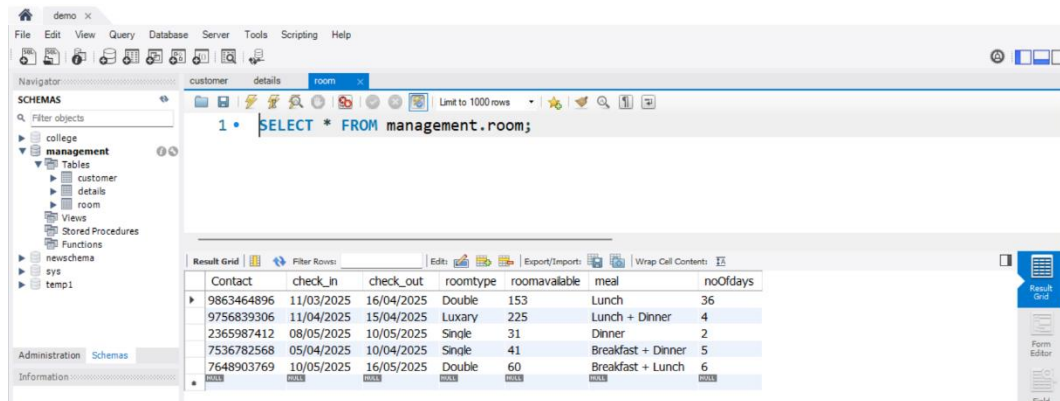
- **Cost Calculation:**

Calculates the total cost of the booking based on room type, meal plan, and number of days, including tax.

Contact	Checkin	Checkout	Room Type	Room No	Meal	NoOfDays
7136782368	23/09/2023	25/09/2023	Single	20	Dinner	2
8954786864	01/10/2023	05/10/2023	Luxury	230	Breakfast	4

- **Database Integration:**

Connects to a MySQL database (Administrate) to store and manage room booking data in the room table.



- **Message:**

1. Input Validation andErrors: `messagebox.showerror("Error", "All fields are required", parent=self.root)`. Shown when required fields (like floor or room type) are missing while adding a new room.
2. `messagebox.showerror("Error", "Please enter valid Room No", parent=self.root)`. Shown when trying to update room details without specifying a valid room number.
3. Operation Success: `messagebox.showinfo("Success", "Room added Successfully", parent=self.root)`. Shown when a room is successfully added to the database.
4. `messagebox.showinfo("Update", "Room Details has been updated seccessfully", parent=self.root)`. Shown when room details are successfully updated.

5. Warnings and Exceptions: `messagebox.showwarning("Warning", f"Something went wrong: {str(es)}", parent=self.root)`. Shown if an exception occurs during the add operation, displaying the error message.
6. **Delete Confirmation:** `messagebox.askyesno("Hotel Administrate system", "Do you want delete this Room Details", parent=self.root)`. Asks the user for confirmation before deleting a room record.
7. **Access Control:** `messagebox.showerror("Access Denied", "You are not authorized to access this page.", parent=self.root)`. Shown if the admin login credentials are incorrect.

This file provides a complete interface for managing hotel room bookings, including data validation, cost calculation, and integration with customer records.

5)Room Details:

This file implements the **Room Details Administrate** module for the hotel Administrate system using Python's Tkinter GUI library and MySQL database. It is designed for admin use and provides the following features:

- **Admin Login:**
Only users with the correct admin username and password can access the room details Administrate interface.

- **Room Administrative Form:**

Allows the admin to add new rooms by specifying the floor, room number, and room type (Single, Double, Luxury).

- **CRUD Operations:**

- **Add:** Insert new room records into the database.
- **Update:** Modify existing room details.
- **Delete:** Remove room records from the database.
- **Reset:** Clear the form fields.

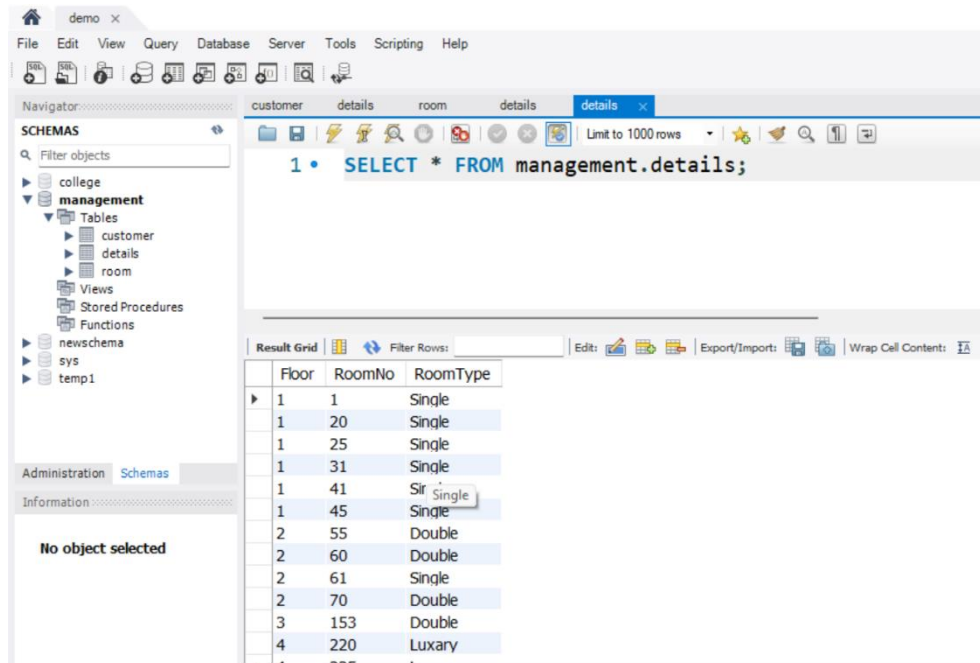
- **Room Details Table:**

Displays all room records in a table with columns for floor, room number, and room type. Selecting a row loads its data into the form for editing.

Floor	Room No	Room Type
2	55	Double
2	60	Double
2	61	Single
2	70	Double
3	153	Double
4	220	Luxury
4	225	Luxury
4	230	Luxury
4	232	Luxury
4	235	Luxury
4	240	Luxury
6	124	Double
8	6	Single

- **Database Integration:**

Connects to a MySQL database (Administrate) and manages data in the details table.



- **Validation and Error Handling:**

Uses message boxes to notify the admin of errors, confirmations, and successful operations.

- This file provides a secure and user-friendly interface for managing the hotel's room inventory, ensuring only authorized users can make changes.

6)About us:

This file implements the **About Us** section of the hotel Administrate system using Python's Tkinter GUI library. It provides information about the hotel, its meal offerings, protocols, and contact details. Key features include:

- **Window Design:**
Displays the hotel's branding with a header image, logo, and a prominent "About Us" title.
- **Meal Details:**
Shows the menu for breakfast, lunch, and dinner in organized frames, listing various dishes available to guests.
- **Hotel Protocols:**
Lists important hotel policies and protocols such as guest safety, registration, secure parking, and emergency response.
- **Contact Information:**
Provides the hotel's phone number, email address, and physical address for guest inquiries.
- **User Interface:**
Uses Tkinter widgets and PIL for images to create an attractive and informative layout.



This file serves as an informational page, helping guests learn more about the hotel's services, rules, and how to get in touch.

